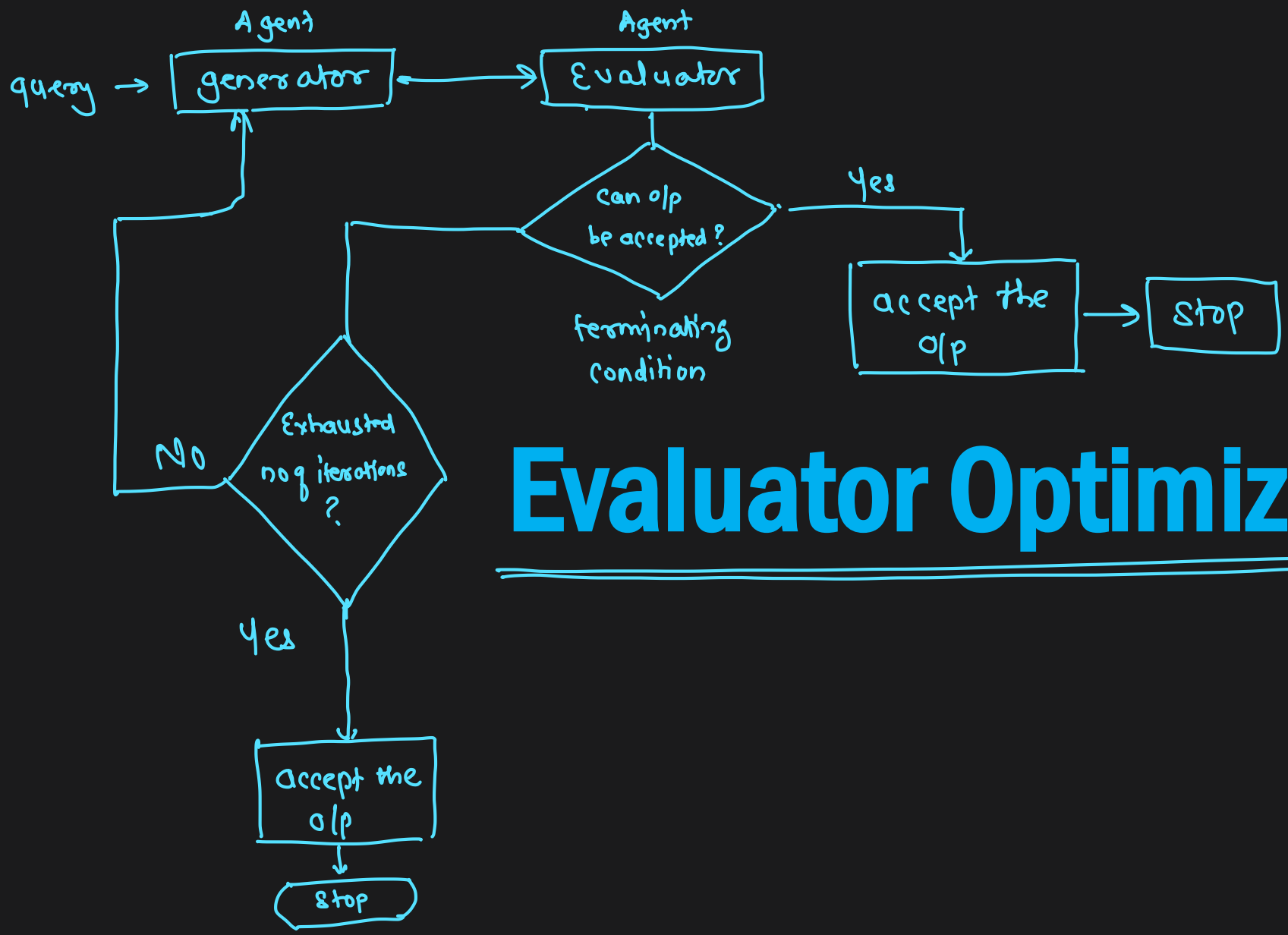


Evaluator Optimizer

Introduction



- The **Evaluator-Optimizer pattern** is a fundamental agentic workflow where two distinct LLM roles operate in an iterative loop
  - a **generator** produces an output
  - an **evaluator** critiques it against defined criteria and feedback is fed back to the generator for refinement
- The cycle repeats until the output meets a quality threshold or a maximum iteration limit is reached
- The key insight is that **generation and evaluation are different cognitive tasks**
- Separating them allows each agent to specialize: the generator focuses on producing content, while the evaluator focuses on applying consistent quality criteria

Structure and Components

- **Generator (Writer)**
  - Produces the initial output and subsequent revisions
  - Takes the original task plus any accumulated feedback as input
  - Can be the same model as the evaluator (with different system prompts) or a different model
- **Evaluator (Critic/Editor)**
  - Applies predefined criteria to assess the generator's output
  - Returns structured feedback: a **verdict** (e.g., PASS / NEEDS_IMPROVEMENT) and **specific issues** to address
  - Must produce **machine-checkable** or at least clearly actionable feedback for the loop to converge
- **Termination Condition**
  - **Primary:** Evaluator returns PASS (quality threshold met)
  - **Fallback:** Maximum iteration count reached, then escalate or return best-effort output
  - A starting limit of **3 iterations** is common, though the right cap depends on task complexity and cost budget

How It Works



- **Initial Generation:** The writer creates a draft based on the given topic or task
- **Evaluation:** The evaluator reviews the draft against criteria such as clarity, engagement, structure, or domain-specific requirements
- **Iterative Refinement:** If improvements are needed, the feedback is incorporated, and the writer generates a revised draft
- **Loop Until Approved:** The process repeats until the content meets the desired quality standards or a maximum number of iterations is reached
- A concrete implementation might enforce **at least one feedback cycle** to demonstrate the pattern, even when the first draft appears acceptable

Evaluator Design Choices



- The evaluator role can take one of two forms, with distinct trade-offs
- Same model, different system prompt
  - Cost: Lower
  - Independence: Limited
  - Risk: Evaluator may inherit generator's blind spots
- Different model
  - Cost: Higher
  - Independence: Full
  - Risk: Independent perspective; useful when roles require different strengths

When to Apply



- The pattern produces measurable improvement under three conditions:
- **"Good output" can be described precisely enough** that the evaluator can score it consistently
- **Iterative refinement genuinely improves quality** — the generator can act on the evaluator's feedback to produce better results
- **The task does not have a single correct answer** that would make iteration redundant
- **Strong Use Cases**
  - **Code generation** with specific requirements (security standards, performance benchmarks, style guidelines)
  - **Professional communications** where tone and precision matter
  - **API documentation generation** (generator writes docs, evaluator checks for completeness, clarity, and accuracy against the codebase)
  - **SQL query production** (generator writes query, evaluator checks for efficiency and security issues)
  - **Literary translations** capturing nuanced meanings
  - **Any scenario where first-draft quality consistently falls short of requirements**

When the Pattern Backfires

- Shared Blind Spots
- Vague Criteria
- Non-Actionable Feedback
- Tasks with a Single Correct Answer
- Resource-Constrained or Real-Time Scenarios



Evaluator Optimizer vs Critic

- The **Evaluator-Optimizer** pattern is defined by its **loop structure**
  - A generator produces output, an evaluator critiques it, and the generator revises repeating until quality passes
  - It is fundamentally about **iterative refinement over time**
- The **Critic** pattern is defined by its **role separation**
  - A dedicated agent whose sole job is to find flaws in another agent's work, operating with its own system prompt, context, and possibly a different model
  - It is fundamentally about **independent verification**

Dimension	Evaluator-Optimizer	Critic Pattern
Primary axis	Temporal (iteration over rounds)	Spatial (separation of roles)
Core mechanism	Feedback loop with revision	Adversarial review without necessarily iterating
Required components	Generator + Evaluator + Loop controller	Critic agent (can exist standalone)
Termination	Explicit (PASS verdict or round cap)	Implicit (critique delivered)
Feedback flow	Critique → Generator revises → Re-evaluate	Critique → (optional) Human or downstream decision

① ingestion

② Retrieval

③ Evaluator

